VIT Bhopal Cycle Sharing System

A Command-Line Cycle Rental Manager

Project Report

Name	Ankush Singh
Registration No.	26BCE10392
Course	Python Essentials (VITyarthi)
Institution	VIT Bhopal
Repository	https://github.com//cycle-rental-system

1. Introduction

Hostels at VIT Bhopal are far from the academic blocks. Walking from the boys' hostels to AB1 takes 15 minutes or more, so a shared cycle service on campus saves students a lot of time. But such a service only works if somebody keeps track of where the cycles are and what each trip should cost.

This project is a command-line program written in Python that manages this job. It keeps a live count of cycles at every hostel and campus location, books rides with a distance-based fare, keeps the ride history and earnings, and reports stations that need cycles moved. All data is saved to a JSON file, so nothing is lost between runs. The program uses only the Python standard library, so it runs anywhere Python 3.8 or newer is installed.

The objectives of the project are:

1. Keep an accurate count of cycles at all 14 stations, with the total always 330, because a ride only moves a cycle.
2. Book rides between any two stations with a fare calculated from the distance using the four slabs (Rs.5, Rs.10, Rs.15, Rs.20).
3. Enforce the ride rules: no ride from an empty station, no ride to a full station (60 cycles).
4. Keep a ride history and give earnings and station reports, so the service can be managed and cycles rebalanced.
5. Save the data after every ride and load it on the next start, with validation and clear error messages instead of crashes.
6. Use only the Python standard library and cover the core logic with unit tests.

2. Problem Statement

The campus has 11 hostels (9 boys' blocks and 2 girls' blocks) with 30 cycles each, giving 330 cycles in total, and 3 campus locations (AB1, AB2 and the Lab Complex) that start empty. Cycles are taken and returned all day, so without a system the counts go wrong quickly: cycles pile up at the academic blocks, hostels run out in the morning, and fares are charged inconsistently.

The program must therefore: (a) keep an accurate count of cycles at all 14 stations, where the total is always 330 because a ride only moves a cycle; (b) allow a ride between any two stations with a fare computed from the distance in four slabs of Rs.5, Rs.10, Rs.15 and Rs.20; (c) reject rides that break the rules (no cycle at the pickup, or the drop station is full at 60 cycles); and (d) keep a history and give reports so the service manager can rebalance the cycles.

3. Functional Requirements

The system is organised into three functional modules, plus input and output behaviour shared by all of them.

Module 1: Station and fare data (stations.py, fares.py)

- Hold the names of all 14 stations and the short codes for them (B3, G1, LAB and so on).
- Convert whatever the user types (b8a, block 8a, 3) into the proper station name, and reject unknown input.

- Give the distance in metres between any two stations, in both directions, and reject a ride from a station to itself.
- Compute the fare from the distance using the four slabs (Rs.5 up to 300 m, Rs.10 up to 600 m, Rs.15 up to 1000 m, Rs.20 above).

Module 2: Ride management (system.py)

- Start every hostel with 30 cycles and the locations with 0, total 330.
- Book a ride: check the rules, move one cycle from pickup to drop, and record student, route, distance, fare and time in the history.
- Reject a ride when the pickup station has no cycle or the drop station is full (60 cycles), without changing any count.
- Keep the total at 330 at all times; a ride never creates or destroys a cycle.
- Save the counts and history to data/cycle_data.json after every ride, and load them on the next start, recovering safely from a damaged file.

Module 3: Reporting and analytics (main.py menu options 1, 4, 5, 6)

- Show the cycle count at every station with the total.
- Show the full ride history with time, route and fare.
- Show total rides and total earnings.
- List hostels running low (under 10 cycles) and crowded stations (over 45), so cycles can be rebalanced.

Input / output structure

Input comes from the keyboard through a numbered menu (options 0-7) or a --demo flag that runs sample rides without typing. Output is printed to the terminal: tables, confirmations, fare previews before booking, and clear error messages when something is not allowed.

4. Non-Functional Requirements

Requirement	How the project meets it
Usability	A numbered menu with plain-English messages; short station codes (b8a, g1, lab) instead of full names; a fare preview before every booking; a --demo mode to try the program without typing.
Reliability	Every ride is saved to JSON immediately, so a crash or Ctrl+C loses nothing; a damaged save file is detected and the system starts fresh instead of crashing.
Performance	All operations are dictionary lookups and small loops, so a menu action answers instantly even with a long ride history.
Error handling	Invalid input (unknown station, empty name, bad rating of distances) is caught and printed as a readable message; the ride rules are checked before anything is changed.
Portability	Standard library only (json, os, datetime, sys, unittest); runs on Windows, macOS and Linux with Python 3.8+ and no installation step.
Maintainability	Fare rules, distances and station names sit in their own files and can be changed without touching the program logic; every module has comments; 15 unit tests protect the behaviour.

5. System Architecture

The program uses a simple three-layer design: the menu (user interface), the CycleSystem class (logic) and the data files (station and fare tables, saved rides). The menu never changes counts directly; it always asks the CycleSystem, which is why the rules cannot be bypassed.

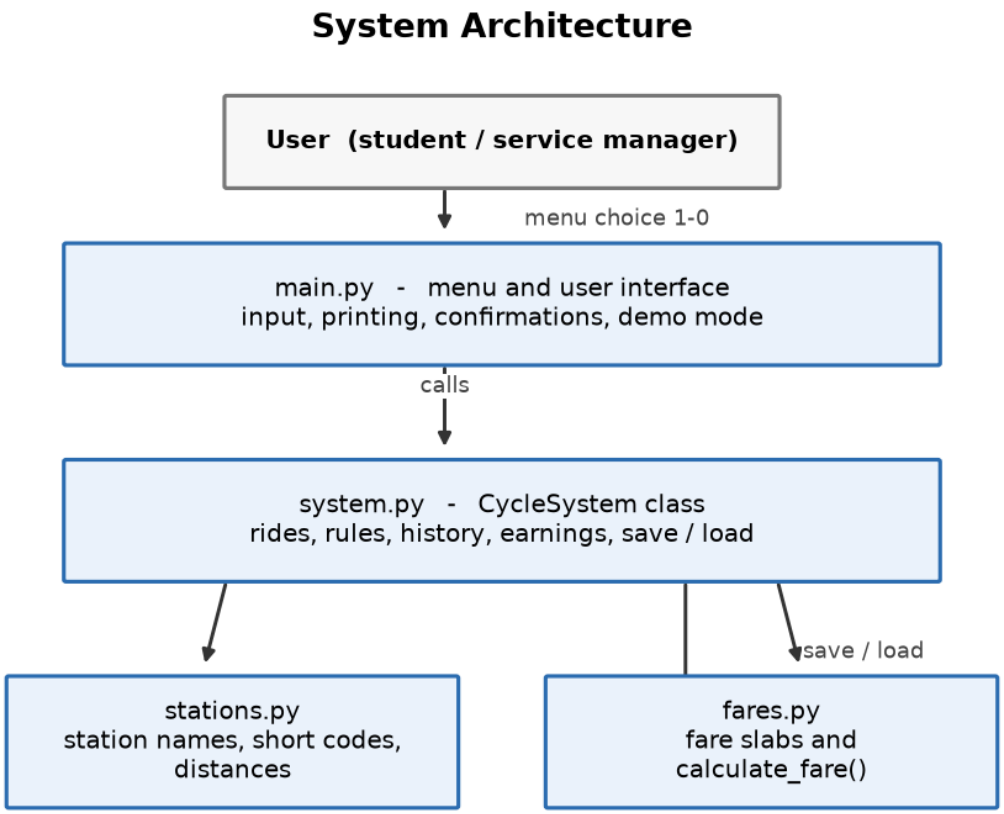


Figure 1: System architecture - layers and modules

6. Design Diagrams

Use case diagram

Two actors use the system. The student mainly takes rides; the service manager (or hostel administration) also reads the reports and can reset the system. All views (counts, fare chart, history, earnings, station report) are available to both.

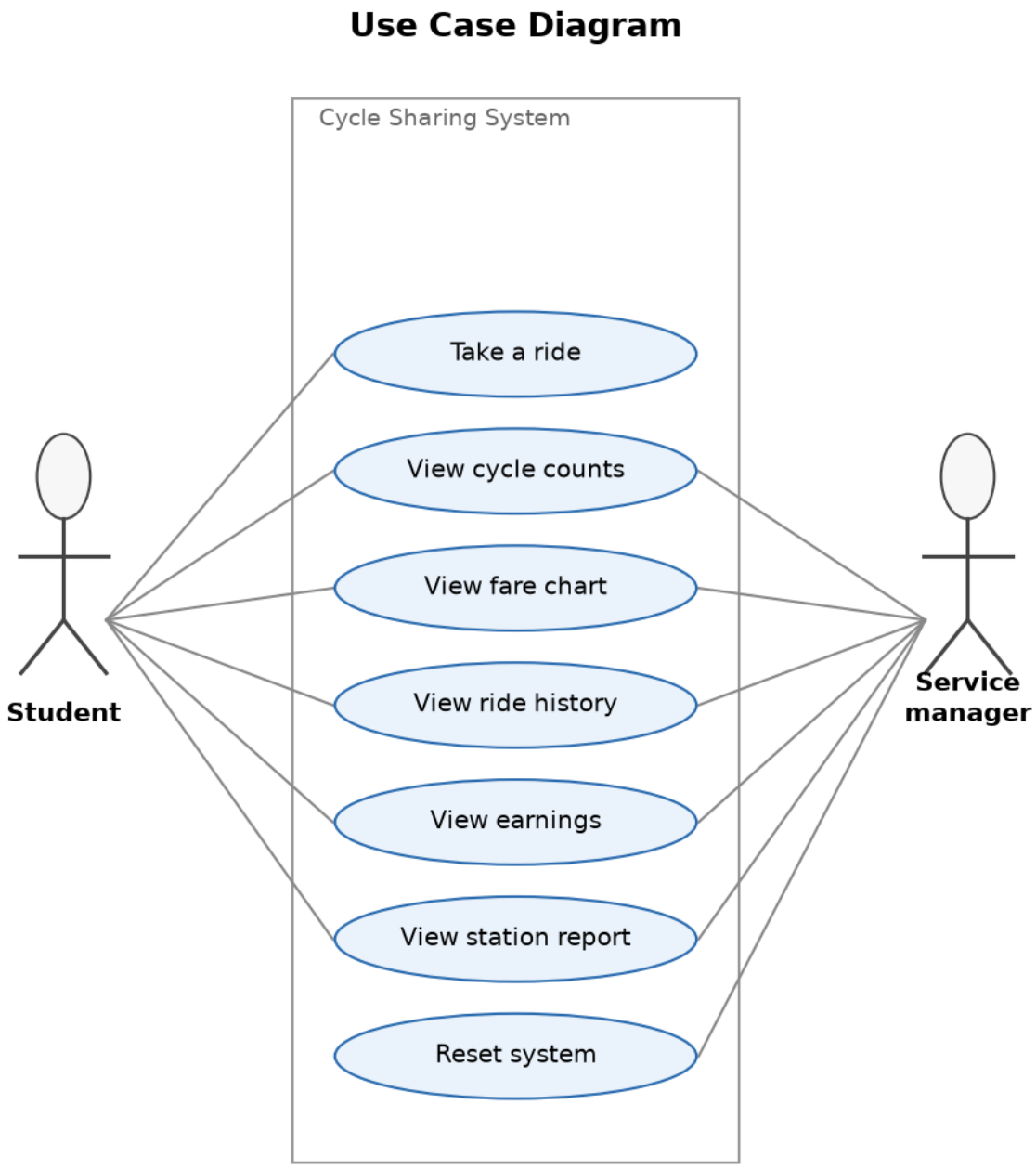


Figure 2: Use case diagram

Workflow diagram

Taking a ride is the central workflow. The program asks for the details, checks the station names, shows the fare and waits for confirmation, then checks the ride rules before finally moving the cycle and saving.

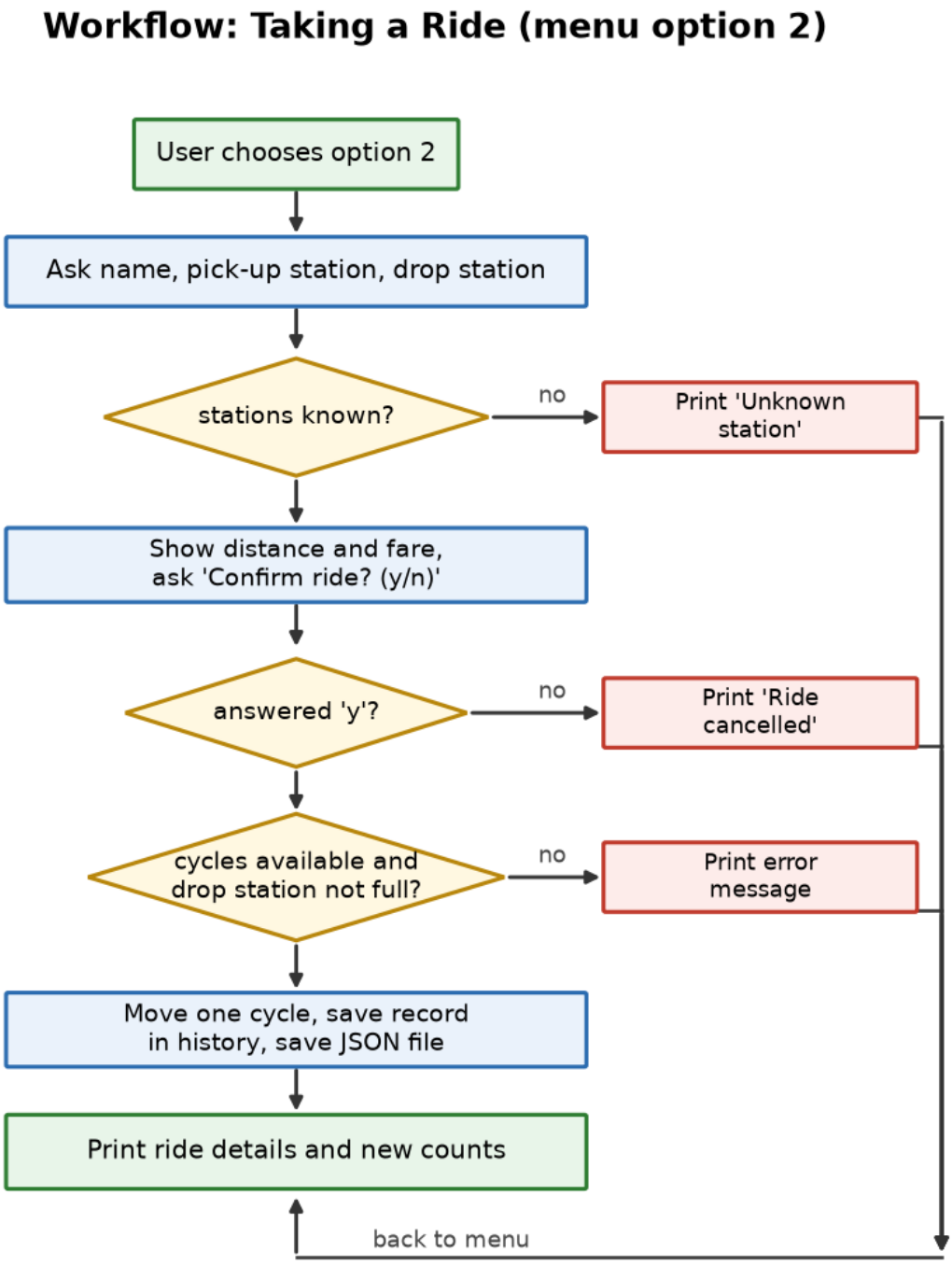


Figure 3: Workflow of taking a ride (menu option 2)

Sequence diagram

The sequence below shows one booking between the user, the menu, the CycleSystem class and the station/fare data.

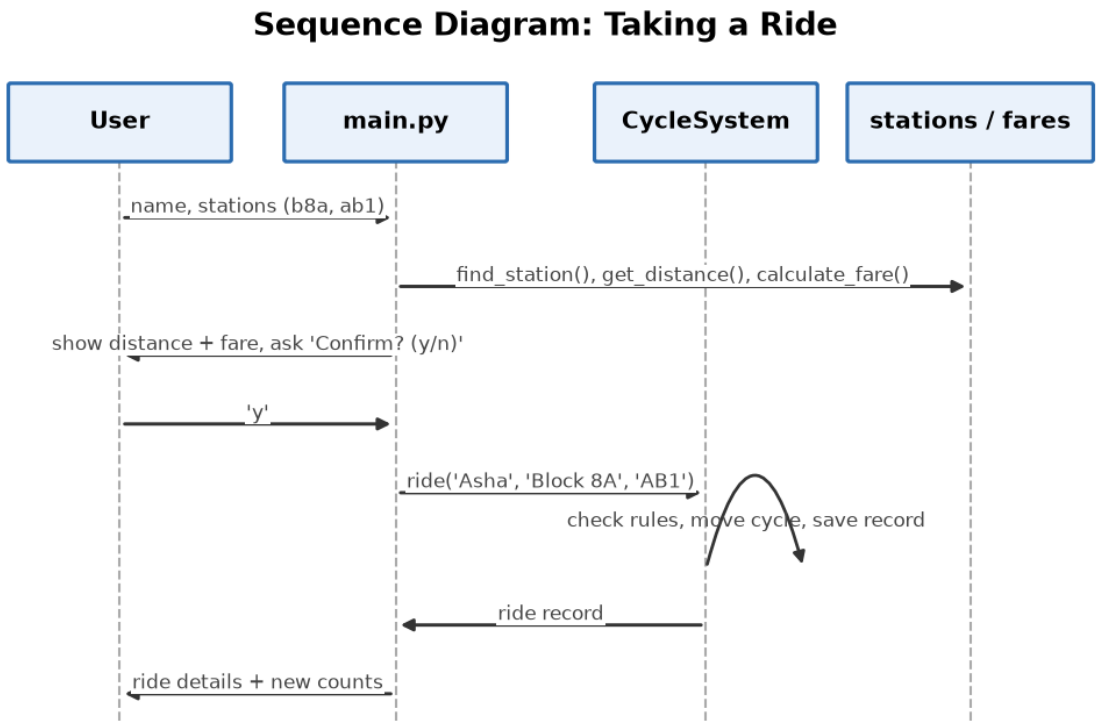


Figure 4: Sequence diagram for taking a ride

Class / module diagram

CycleSystem is the only class; stations and fares are plain modules of functions and tables. main.py creates the CycleSystem object and drives it from the menu.

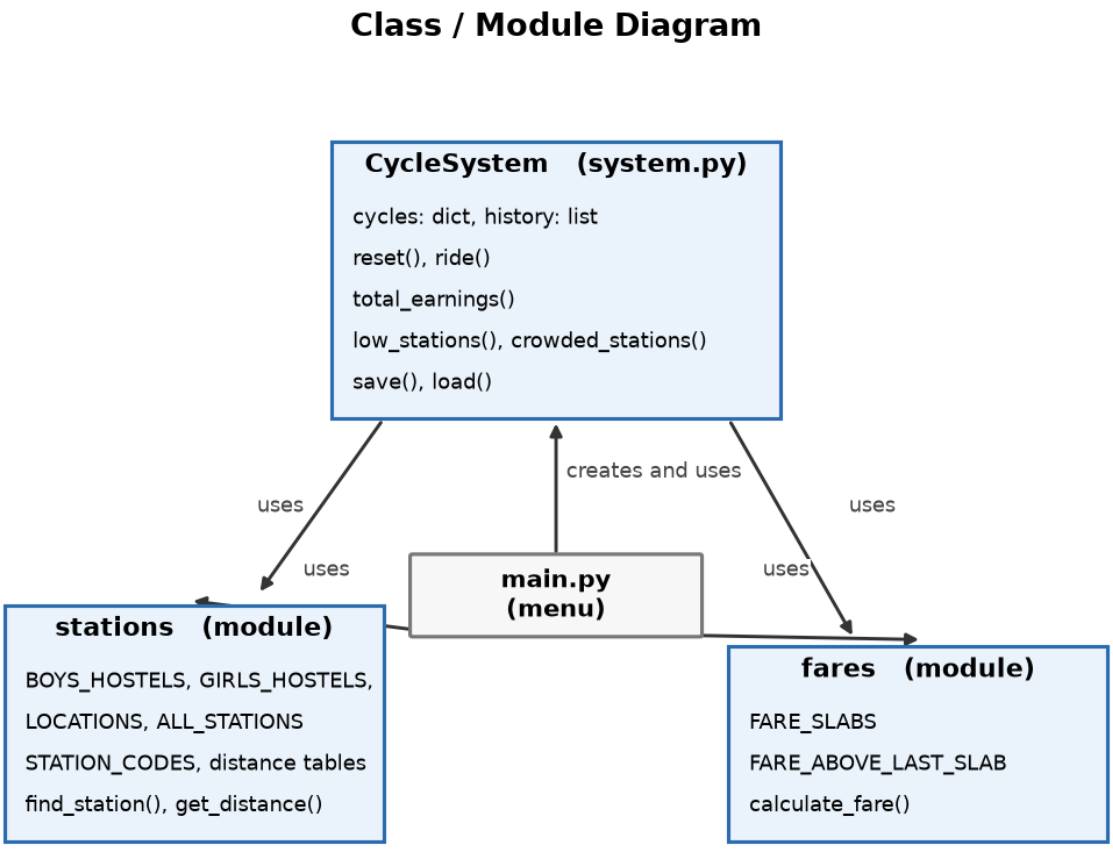


Figure 5: Class and module diagram

Storage design

There is no database; all data lives in one JSON file, data/cycle_data.json, with the following schema:

```
{
  "cycles":  { "Block 1": 30, ..., "AB1": 0, "AB2": 0, "Lab Complex": 0 },
  "history": [
    { "ride_no": 1, "student": "Asha", "from": "Block 8A",
      "to": "AB1", "distance": 1600, "fare": 20,
      "time": "29-09-2026 21:15" },
    ...
  ]
}
```

cycles maps every station name to its count (an integer), and history is a list of ride records. On load, the program checks that every station exists with an integer count before using the data; a file that fails this check is discarded and the system starts fresh.

7. Design Decisions and Rationale

Decision	Why
A single CycleSystem class instead of separate classes per station	All stations behave identically, so one dictionary of counts plus the ride rules covers everything; per-station classes would add code without adding behaviour.
Distances and fares in separate files from the logic	The prices and campus distances change more often than the logic; keeping them in stations.py and fares.py means an edit to a number can never break a rule.
Fare computed from distance slabs, not stored per route	There are 14 x 13 = 182 ordered routes; four slabs replace 182 entries and stay correct if a distance value is corrected.
Menu confirms the fare before booking	The student sees the price first and can cancel; this also gives the manager a way to quote prices without taking a ride.
JSON instead of a database	One small file, human-readable, part of the standard library, and enough for the data volume; a database would add setup for no gain here.
Saving after every ride, not on exit only	If the terminal closes or the power fails, at most the action in progress is lost, not the whole session.
Rejected rides raise a ValueError	The rule check and the error message live in one place (system.py), and both the menu and the demo can catch the same error the same way.

8. Implementation Details

The project has four Python modules plus tests. stations.py holds the names, codes and distance tables and turns user input into a proper station name with find_station(). fares.py holds the slab table and calculate_fare(). system.py contains the CycleSystem class with the ride rules, history, earnings and save/load. main.py is the menu: it prints the options, reads the input, calls the system and prints the results. tests/test_cycle.py contains the unit tests.

Key points of the implementation:

1. find_station() normalises the input (strip, upper case, remove spaces) and tries three things in order: the short-code table, the full station names, and a plain number ('3' meaning Block 3). It returns None for unknown input, and the caller prints the error.
2. get_distance() separates four cases: hostel to hostel (simple rules, plus the Block 8A/8B exception), hostel to location (the given table), location to hostel (same table reversed) and location to location (a small pair table). Because every table is symmetric, the program checks both orders so distance A to B always equals distance B to A.
3. ride() checks the two rules (a cycle must be available, the drop station must not be full) before touching any count, so a rejected ride changes nothing. Only then does it move the cycle and append a record with a timestamp from datetime.
4. save() writes the counts and history with json.dump; load() validates that every station is present with an integer count, and treats a broken file as 'start fresh' rather than an error.
5. The menu is one while loop with if/elif branches; option 7 (reset) asks for confirmation because it erases the ride history.

9. Results

The program was run through its demo mode (python main.py --demo), which books six rides and then attempts a seventh that must fail, because the Lab Complex has no cycle at that point. The output:

```
Asha: Block 8A -> AB1, 1600 m, Rs.20
Ravi: Block 1 -> AB2, 750 m, Rs.15
Meena: AB1 -> Block 8A, 1600 m, Rs.20
Karan: Girls Block 1 -> Lab Complex, 850 m, Rs.15
Zoya: Block 2 -> Block 3, 400 m, Rs.10
Dev: Lab Complex -> Block 1, 1850 m, Rs.20
Neha: Lab Complex -> Girls Block 2 failed: No cycles available at Lab Complex.
Station         Cycles
----------------------
Block 1         30          Girls Block 1   29
Block 2         29          Girls Block 2   30
Block 3         31          AB1              0
Block 4         30          AB2              1
...                         Lab Complex      0
----------------------
Total           330
Total rides   : 6
Total earnings: Rs.100
```

Every rule shows in this short run: the total stays at 330, the failed ride changes nothing, and the fares match the slabs (750 m gives Rs.15, 400 m gives Rs.10, and so on). Screenshots of the menu, a booked ride, the fare chart and the test run are included in the screenshots/ folder of the repository.

10. Testing Approach

The project has 15 unit tests in tests/test_cycle.py, run with python -m unittest discover -s tests -v. They cover:

- Fare slab edges: 300/301 m, 600/601 m and 1000/1001 m (the exact borders between Rs.5, Rs.10, Rs.15 and Rs.20).
- Distances: the values given in the problem, distance A to B equal to B to A for all 182 ordered pairs, boys hostels within 1350-1650 m of AB1, and errors for unknown or same stations.
- Station parsing: short codes, full names in any case, plain numbers, and rejection of unknown input.
- Rides: starting counts (30/30/.../0, total 330), a ride moving exactly one cycle, the total staying 330 after several rides, rejection when no cycle is available or the station is full (with counts unchanged), and correct earnings.
- Storage: saving to a file and loading it into a fresh system gives back the same counts and history.

All 15 tests pass:

```
$ python -m unittest discover -s tests
...............................
```

```
----------------------------------------------------------------------
Ran 15 tests in 0.005s

OK
```

Manual testing complemented the unit tests: the demo run, an unknown-station input, an empty name, refusing the fare confirmation, resetting with and without confirmation, ending input with Ctrl+C (data survives), deleting the save file, and hand-editing it into a damaged state to check that the program starts fresh instead of crashing.

11. Challenges Faced

1. Keeping the distances symmetric. My first distance tables only worked in one direction, so riding Block 1 to AB1 and AB1 to Block 1 could give different fares. I fixed it by checking both key orders in get_distance() and added a test that compares every ordered pair, so this cannot come back silently.
2. Free-form user input. Users type 'b8a', 'B8A', 'block 8a' or just '3'. Normalising the input (strip, upper case, remove spaces) and then trying three lookups in order made all of these work, and unknown input returns None instead of raising an error in the middle of a menu action.
3. The fare slab borders. Off-by-one mistakes are easy here: does 300 m cost Rs.5 or Rs.10? I wrote the tests for the exact borders (300/301, 600/601, 1000/1001) before trusting the fare code.
4. A damaged save file. A JSON file can be cut off mid-write. Instead of crashing, load() validates that every station exists with an integer count and falls back to a fresh start, which also keeps the total at 330.
5. Losing rides on exit. Saving only when the user presses 0 loses everything on a crash, so I moved the save to right after every ride and to the Ctrl+C handler.

12. Learnings and Key Takeaways

Building this project turned the course topics into something I could see working. The main takeaways:

- Designing with dictionaries: station counts, codes and distance tables all became dict lookups, which is why the program is fast and short.
- Classes and separation of concerns: putting the rules inside CycleSystem and keeping the menu dumb meant the rules cannot be bypassed and testing is easy.
- Exception handling as a design tool: ValueError with a clear message is an interface, not an accident; the menu, the demo and the tests all rely on it.
- Test-driven confidence: writing tests for the fare borders and the ride rules caught my off-by-one mistakes and made every later change safe.
- File handling and JSON: persisting state, validating it on load, and degrading gracefully when it is broken.
- Version control: committing in small steps made it possible to go back when a change broke something.

13. Future Enhancements

1. Track individual cycles by number and their condition, not just counts per station.
2. A time-of-day or distance-based membership pricing option on top of the slabs.
3. Move the data to SQLite and support several terminals at the same time.
4. Read the live counts from station QR scans instead of typed input.
5. A simple rebalancing suggestion: which station to move cycles from and to, based on the low/crowded report.
6. Export the ride history and earnings to CSV for accounting.

14. References

- Python 3 documentation, docs.python.org (built-in types, json, datetime, unittest modules).
- VITyarthi Python Essentials course material (course notes and examples).
- The campus distance and fare data given in the project problem statement.